3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

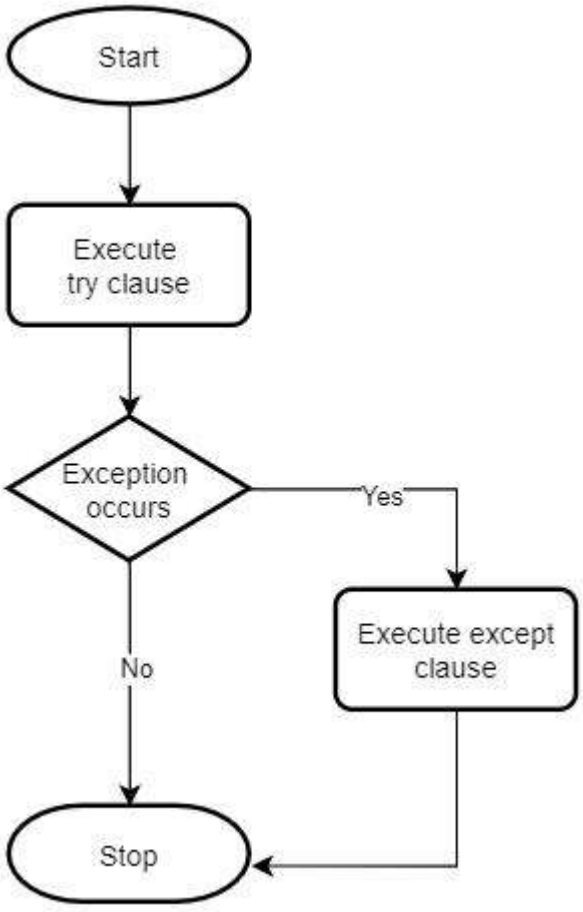
Classes : Offline & Online

Daily Teaching : 2 Hours

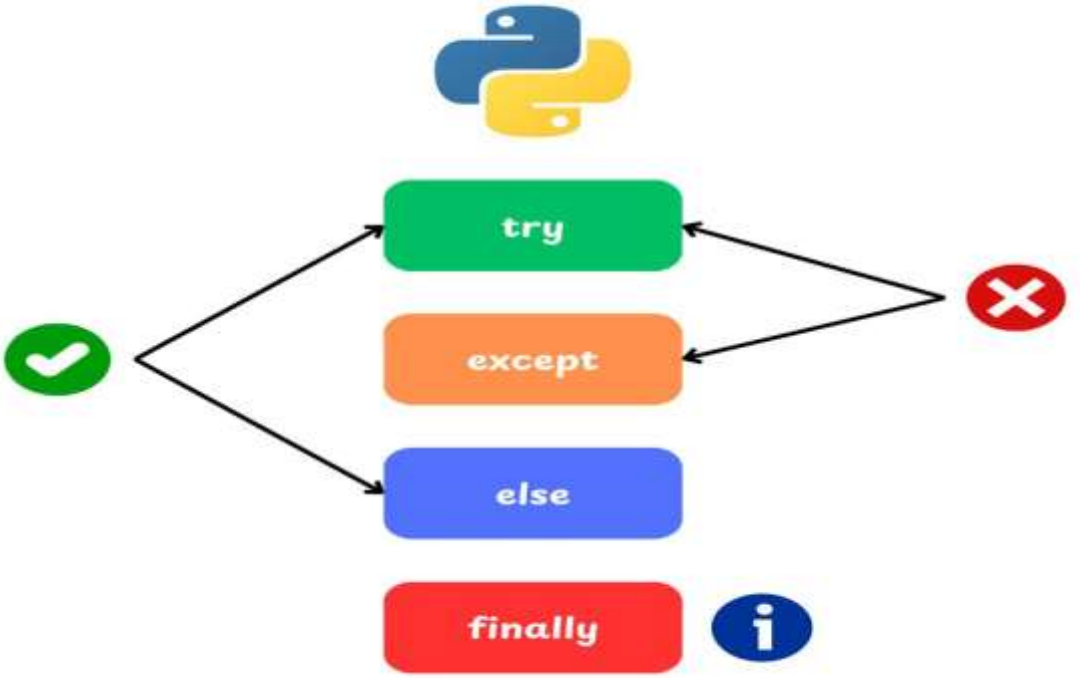
Total Classes : 100

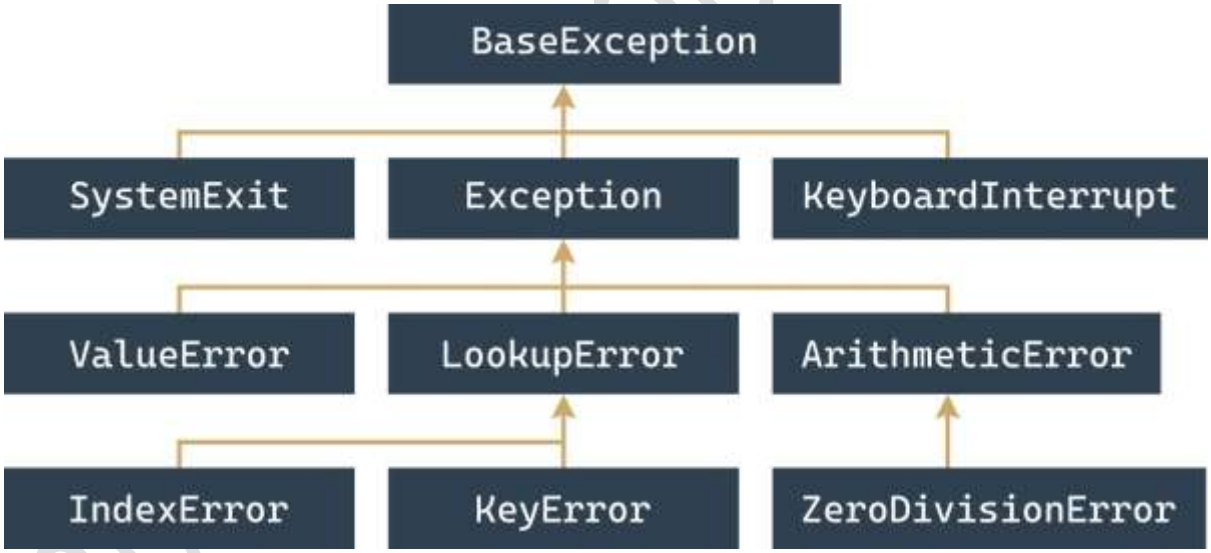
Level 4 – AI Professional Foundation(11th–12th Standard)

Class 17 – Exception Handling



3DWEBSOFT FOUNDATION





1. Learning Objectives

By the end of this lesson, students will be able to:

- Understand errors and exceptions in Python.
- Differentiate between **syntax errors** and **runtime exceptions**.

- Understand try, except, else, and finally.
- Handle common Python exceptions.
- Use multiple except blocks.
- Raise exceptions using raise.
- Create custom exceptions.
- Build applications that do not crash unexpectedly.
- Apply exception handling to real-world programs.

2. What is an Error?

An **error** is a problem in a program that prevents the program from working correctly.

For example:

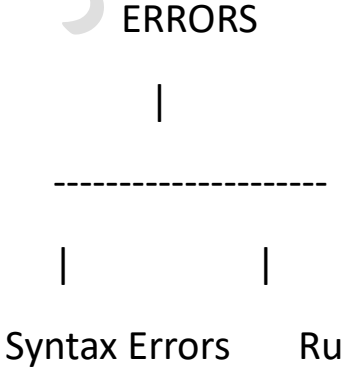
print("Hello"

This program has a syntax problem because the closing) is missing.

Python reports an error instead of executing the program.

3. Types of Errors

Python errors can broadly be understood as:



3DWEBSOFT FOUNDATION

|

Exceptions

A. Syntax Error

Occurs when Python rules are violated.

```
if 10 > 5
   print("Yes")
```

The colon : is missing.

Python will report a SyntaxError.

B. Runtime Error / Exception

The program starts running, but something unexpected happens.

Example:

```
number = 10
result = number / 0
```

Output:

ZeroDivisionError

4. What is an Exception?

An **exception** is an event that occurs during program execution and interrupts the normal flow of the program.

Example:

```
age = int(input("Enter your age: "))
```

If the user enters:

abc

SUWEBSOFT FOUNDATION

Python cannot convert "abc" into an integer.

It produces:

ValueError

Without exception handling, the program may stop.

5. Why Exception Handling is Important

Imagine a student-management application.

The program asks:

Enter student age:

The user enters:

seventeen

Without exception handling:

Traceback...

ValueError...

The application may terminate.

With exception handling:

Invalid age!

Please enter a number.

The application can continue.

Main purpose

Exception handling makes programs safer, more reliable and user-friendly.

SUWEBSOFT FOUNDATION

6. Common Python Exceptions

Exception	When it occurs
ValueError	Invalid value
TypeError	Wrong data type
ZeroDivisionError	Division by zero
IndexError	Invalid list index
KeyError	Missing dictionary key
FileNotFoundError	File does not exist
NameError	Variable does not exist
AttributeError	Invalid attribute/method
ImportError	Import problem
Exception	General exception

7. Simple Example of an Exception

```
number = int(input("Enter a number: "))


print("Number:", number)
```

If the user enters:

25

Output:

Number: 25

3DWEBSOFT FOUNDATION

But if the user enters:

hello

Python raises:

ValueError

8. The try Block

The try block contains code that **might produce an exception**.

Syntax:

```
try:
    # risky code
```

Example:

```
try:
    number = int(input("Enter a number: "))
    print(number)
```

But try normally needs an except or another appropriate handler.

9. The except Block

The except block handles an exception.

Syntax:

```
try:
    risky_code
except:
    error_handling_code
```

3DWEBSOFT FOUNDATION

Example

```
try:
    number = int(input("Enter a number: "))
    print("You entered:", number)
except:
    print("Invalid input!")
```

Input

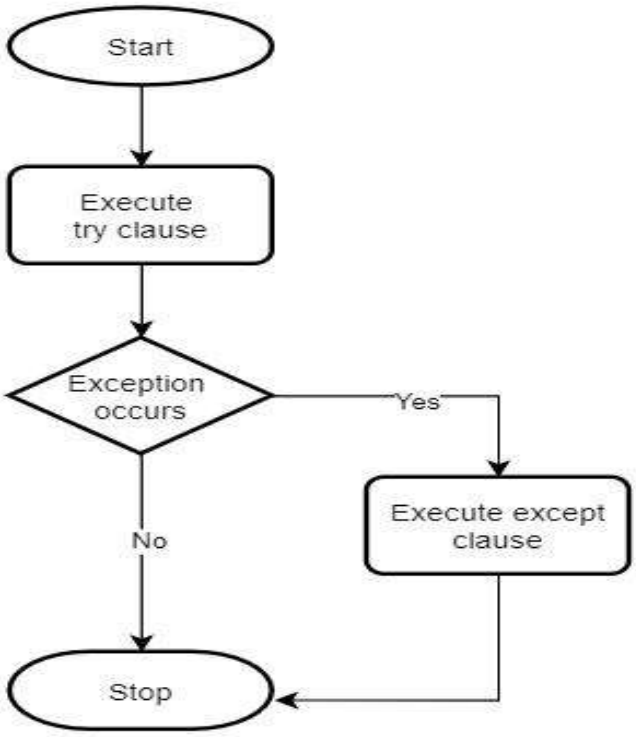
Enter a number: abc

Output

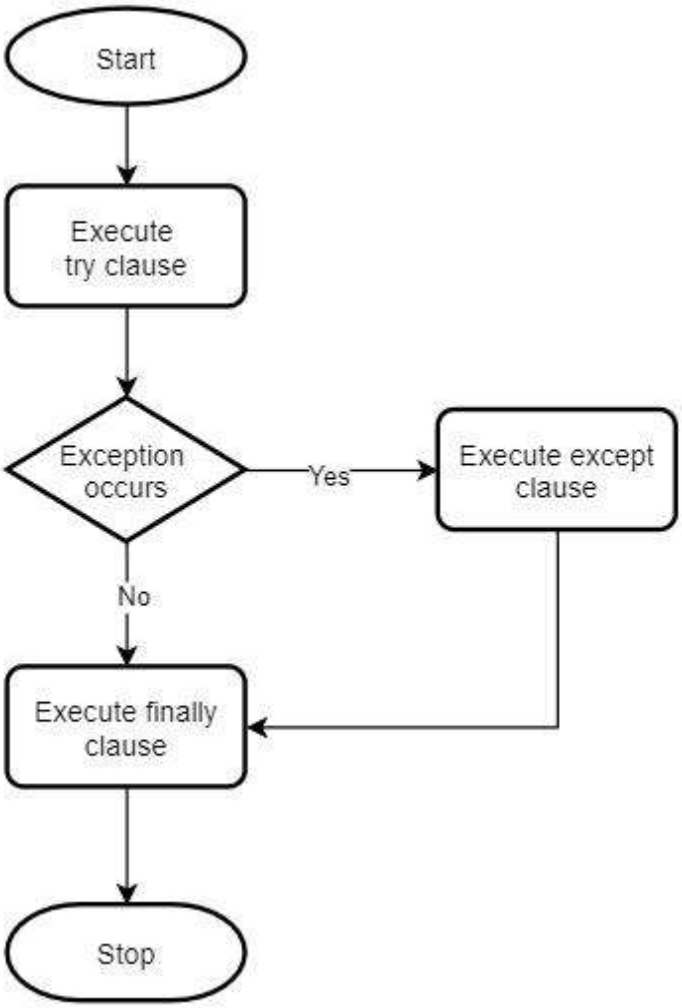
Invalid input!

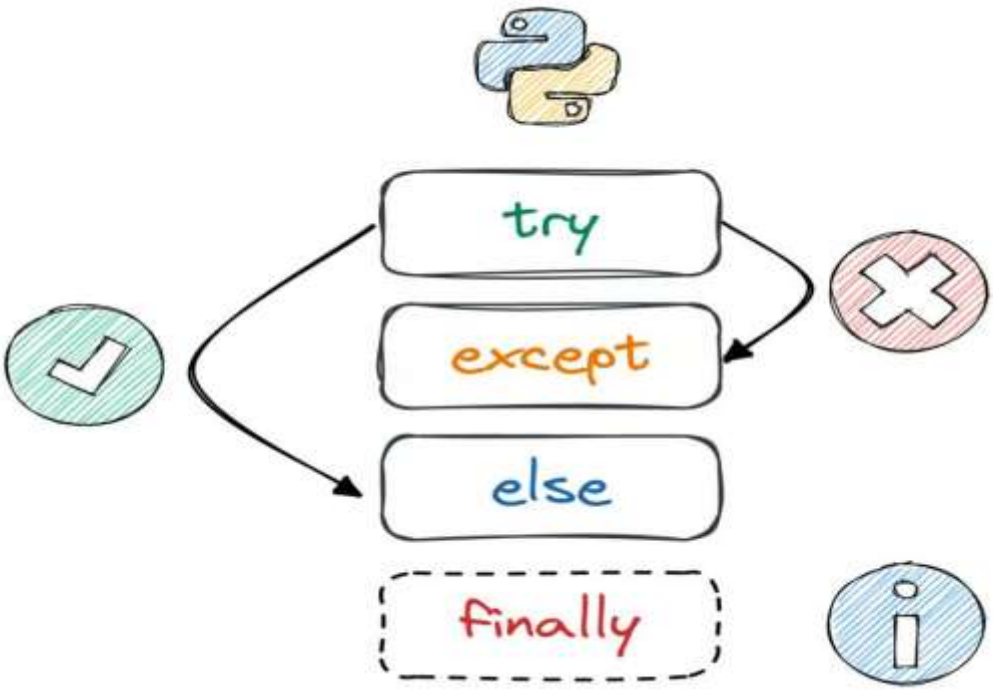
The application does not crash with an unhandled exception.

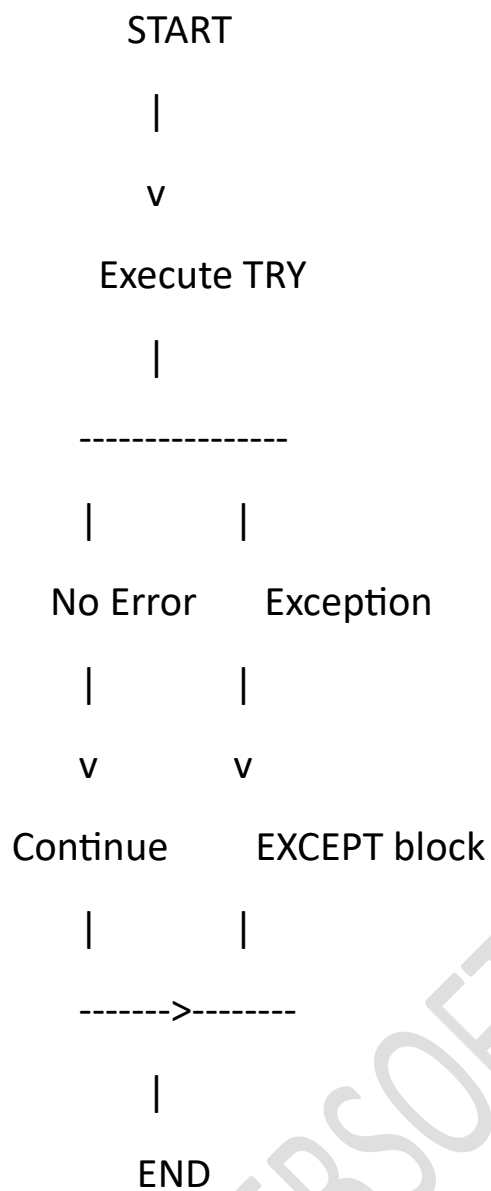
10. try-except Flow



3DW SOFT FOUNDATION


Start
Execute try clause
Exception occurs
Yes
Execute except clause
No
Execute finally clause
Stop


try
except
else
finally



9JWEBSOFT FOUNDATION

11. Handling a Specific Exception

It is better to specify the type of exception.

```
try:
    number = int(input("Enter a number: "))

except ValueError:
    print("Please enter a valid number.")
```

Input

Enter a number: abc

Output

Please enter a valid number.

12. Handling ZeroDivisionError

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))

    result = a / b

    print("Result:", result)

except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Example

Enter first number: 20

Enter second number: 0

Cannot divide by zero.

13. Handling Multiple Exceptions

A program can have multiple except blocks.

3DWEBSOFT FOUNDATION

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    result = a / b
    print("Result:", result)
except ValueError:
    print("Please enter numbers only.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Why?

Different problems need different solutions.

Wrong input → ValueError

Zero divisor → ZeroDivisionError

14. Catching Multiple Exceptions Together

We can also write:

```
try:
    number = int(input("Enter number: "))

except (ValueError, TypeError):
    print("Invalid data.")
```

This is useful when the same response is suitable for multiple exception types.

3DWEBSOFT FOUNDATION

15. The else Block

The else block executes **only when no exception occurs**.

Syntax:

```
try:
    # risky code
except:
    # error
else:
    # successful execution
```

Example

```
try:
    number = int(input("Enter a number: "))
except ValueError:
    print("Invalid number.")
else:
    print("Number accepted:", number)
```

Input

Enter a number: 25

Output:

Number accepted: 25

16. The finally Block

The finally block executes **whether an exception occurs or not**.

SDWEBSOFT FOUNDATION

Syntax:

```
try:
    # code
except:
    # error
finally:
    # always executed
```

Example:

```
try:
    number = int(input("Enter number: "))
    print(number)
except ValueError:
    print("Invalid number.")
finally:
    print("Program execution completed.")
```

Input

Enter number: abc

Output

Invalid number.

Program execution completed.

17. Understanding try, except, else, finally

3DWEBSOFT FOUNDATION

Block	Purpose
try	Code that may cause an exception
except	Handles the exception
else	Runs when no exception occurs
finally	Runs whether exception occurs or not

Easy memory trick

TRY → Try the operation

EXCEPT → Handle the problem

ELSE → Successful operation

FINALLY → Always execute

18. Practical Program – Safe Calculator

```
def calculator():
    try:
        a = float(input("Enter first number: "))
        b = float(input("Enter second number: "))
        operation = input("Enter operation (+, -, *, /): ")
        if operation == "+":
            result = a + b
        elif operation == "-":
            result = a - b
        elif operation == "*":
```

3DWEBSOFT FOUNDATION

```
            result = a * b
        elif operation == "/":
            result = a / b
        else:
            print("Invalid operation.")
            return
        print("Result:", result)
    except ValueError:
        print("Please enter valid numbers.")
    except ZeroDivisionError:
        print("Cannot divide by zero.")
calculator()
```

Example

```
Enter first number: 20
Enter second number: 0
Enter operation (+, -, *, /): /
Cannot divide by zero.
```

19. IndexError

An IndexError occurs when we try to access an invalid list index.

```
numbers = [10, 20, 30]
print(numbers[5])
```

There is no index 5.

3DWEBSOFT FOUNDATION

Python produces:

IndexError

Safe version

```
numbers = [10, 20, 30]
try:
    print(numbers[5])
except IndexError:
    print("Invalid list index.")
```

Output:

Invalid list index.

20. KeyError

A KeyError occurs when a dictionary key does not exist.

```
student = {
    "name": "Rahul",
    "marks": 85
}
print(student["age"])
```

Output:

KeyError

Safe version

```
student = {
    "name": "Rahul",
```

3DWEBSOFT FOUNDATION

```
    "marks": 85
}
try:
    print(student["age"])
except KeyError:
    print("Age information is not available.")
```

21. TypeError

A TypeError occurs when an operation is performed on incompatible data types.

Example:

```
name = "Rahul"
age = 17
print(name + age)
```

This is invalid because Python cannot directly concatenate a string and integer.

Safe version

```
try:
    name = "Rahul"
    age = 17
    print(name + age)
except TypeError:
    print("Cannot combine these data types.")
```

3DWEBSOFT FOUNDATION

22. FileNotFoundError

This is especially important because students learned file handling in previous lessons.

```
try:
    with open("students.txt", "r") as file:
        data = file.read()


except FileNotFoundError:
    print("File does not exist.")
```

Instead of crashing, the application gives a useful message.

23. Exception Handling with JSON

This connects directly with **Class 16 – JSON Data Handling**.

Suppose:

students.json

does not exist.

We can safely handle it:

```
import json
try:
    with open("students.json", "r") as file:
        students = json.load(file)
except FileNotFoundError:
    print("Student database not found.")
```

3DWEBSOFT FOUNDATION

```
    students = []
except json.JSONDecodeError:
    print("JSON file contains invalid data.")
    students = []
```

This is a real-world example of building an **error-safe application**.

24. The raise Statement

Sometimes we want to create an exception ourselves.

Python provides:

raise

Example:

```
age = int(input("Enter age: "))
if age < 0:
    raise ValueError("Age cannot be negative.")
print("Age:", age)
```

Input

Enter age: -5

The program raises:

ValueError: Age cannot be negative.

25. Handling a Raised Exception

We can combine raise with try-except.

```
try:
```

SOWEBSOFT FOUNDATION

```
    age = int(input("Enter age: "))
    if age < 0:
        raise ValueError("Age cannot be negative.")
    print("Age accepted.")
except ValueError as e:
    print("Error:", e)
```

Output

```
Enter age: -5
Error: Age cannot be negative.
```

26. What is as e?

In:

except ValueError as e:

e stores information about the exception.

Example:

```
try:
    raise ValueError("Invalid student age")
except ValueError as e:
    print(e)
```

Output:

```
Invalid student age
```

27. Custom Exceptions

Python allows us to create our own exception classes.

3JWEBSOFT FOUNDATION

Example:

```
class InvalidMarksError(Exception):
    pass
```

Now we can use it:

```
try:
    marks = int(input("Enter marks: "))
    if marks < 0 or marks > 100:
        raise InvalidMarksError("Marks must be between 0 and 100.")
    print("Marks:", marks)
except InvalidMarksError as e:
    print("Error:", e)
```

Input

Enter marks: 150

Output

Error: Marks must be between 0 and 100.

28. Exception Hierarchy

Python exceptions have a hierarchy.

A simplified view:

```
BaseException
     |
  Exception
     |
```

3DWEBSOFT FOUNDATION

```
 ------------------------------
  |      |       |          |
ValueError TypeError IndexError KeyError
           |
     ZeroDivisionError
```

Students don't need to memorize the complete hierarchy at this stage.

The important idea is:

Many specific exceptions belong under the general Exception class.

29. General Exception Handling

We can use:

except Exception as e:

Example:

```
try:
    number = int(input("Enter number: "))
    result = 100 / number
    print(result)

except Exception as e:
    print("An error occurred:", e)
```

Important Teaching Point

3DWEBSOFT FOUNDATION

Do not use a broad except Exception everywhere without understanding the specific problem.

Prefer:

except ValueError:

when you know the expected error.

30. Nested Exception Handling

A try block can exist inside another try block.

```
try:

    number = int(input("Enter number: "))
    try:
        result = 100 / number
        print(result)
    except ZeroDivisionError:
        print("Cannot divide by zero.")
except ValueError:
    print("Please enter a number.")
```

This allows different parts of a program to handle different errors.

31. Practical – Safe Student Marks Program

```
try:
    name = input("Enter student name: ")
```

3DWEBSOFT FOUNDATION

```
    marks = float(input("Enter marks: "))
    if marks < 0 or marks > 100:
        raise ValueError("Marks must be between 0 and 100.")
    print("\nStudent:", name)
    print("Marks:", marks)
except ValueError as e:
    print("Invalid data:", e)
finally:
    print("Student entry process completed.")
```

Example 1

```
Enter student name: Rahul
Enter marks: 85
Student: Rahul
Marks: 85.0
Student entry process completed.
```

Example 2

```
Enter student name: Rahul
Enter marks: 120
Invalid data: Marks must be between 0 and 100.
Student entry process completed.
```

32. Best Practices

1. Catch specific exceptions

SUWEBSOFT FOUNDATION

Prefer:

except ValueError:

instead of unnecessarily using:

except:

2. Give useful messages

Bad:

Error

Better:

Please enter a valid number.

3. Don't hide errors

Avoid silently ignoring problems.

4. Use finally for cleanup

For example, closing resources when needed.

5. Validate user input

Check:

- Number ranges
- Required fields
- File existence
- Correct data types

33. Common Mistakes

Mistake 1

Putting risky code outside try.

3DWEBSOFT FOUNDATION

Incorrect:

```
number = int(input("Number: "))


try:
    print(number)
except ValueError:
    print("Invalid")
```

The conversion happens before the try.

Correct:

```
try:
    number = int(input("Number: "))
    print(number)
except ValueError:
    print("Invalid")
```

Mistake 2

Using a broad except for everything.

```
except:
    print("Something went wrong")
```

This can make debugging difficult.

Mistake 3

Forgetting that finally always runs.

3DWEBSOFT FOUNDATION

```
try:
    print("Hello")
finally:
    print("Always executed")
```

Output:

Hello

Always executed

34. Real-World Applications

Exception handling is used in:

Banking Applications

↓

Payment Systems

↓

Student Management Systems

↓

E-Commerce Applications

↓

Mobile Applications

↓

Web Applications

↓

Data Processing Systems

3DWEBSOFT FOUNDATION

For example, a banking application must safely handle:

- Invalid account numbers
- Invalid amounts
- Insufficient balance
- Network failures
- Database errors

35. Practical Activity – Error Detective

Give students these programs and ask them to identify the exception.

Program A

```
print(10 / 0)
```

Answer: ZeroDivisionError

Program B

```
number = int("hello")
```

Answer: ValueError

Program C

```
numbers = [10, 20, 30]
print(numbers[10])
```

Answer: IndexError

Program D

```
student = {"name": "Rahul"}
print(student["age"])
```

Answer: KeyError

3DWEBSOFT FOUNDATION

Program E

```
with open("abc.txt") as file:
    print(file.read())
```

If the file doesn't exist:

Answer: FileNotFoundError

36. Mini Project – Error-Safe Student Application

Project Name

Error-Safe Student Management System

Students should create an application with:

```
================================
   STUDENT MANAGEMENT SYSTEM
================================

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
```

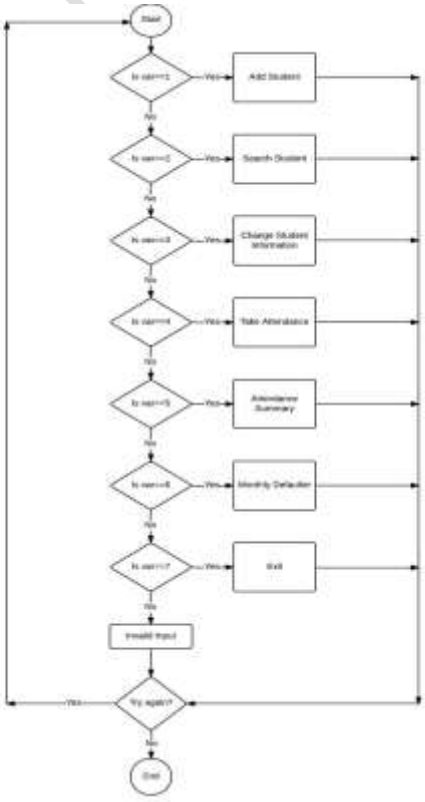
Required Exception Handling

The application should handle:

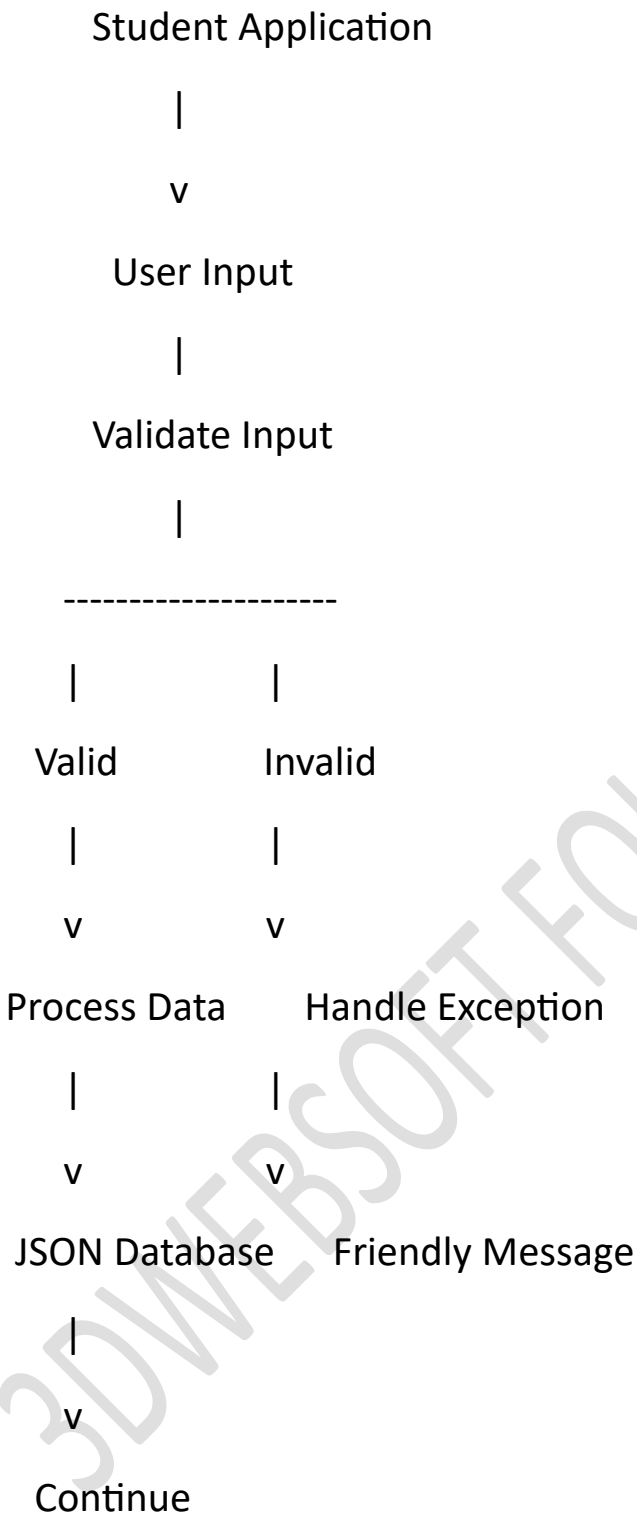
©JWEBSOFT FOUNDATION

Situation	Exception / Handling
User enters text instead of roll number	ValueError
Student not found	Custom message
JSON file missing	FileNotFoundError
Invalid JSON	JSONDecodeError
Invalid marks	ValueError
Duplicate roll number	Validation
Unexpected problem	Appropriate exception handling

Application Flow



3DWF T FOUNDATION



3DWEBSOFT FOUNDATION

37. Student Challenge

Modify the Class 16 JSON Student Database to include:

Validation

```
if marks < 0 or marks > 100:
    raise ValueError("Marks must be between 0 and 100.")
```

File handling

```
try:
    with open("students.json", "r") as file:
        students = json.load(file)
except FileNotFoundError:
    students = []
```

Input handling

```
try:
    roll = int(input("Enter roll number: "))
except ValueError:
    print("Roll number must be a number.")
```

Students will now combine:

JSON + File Handling + Functions + Exception Handling

38. Quick Revision

```
Exception Handling
      |
      +— try
      |    ↓
      |  Risky Code
      |
```

OJWEBSOFT FOUNDATION

+— except

| ↓

| Handle Error

|

+— else

| ↓

| No Error

|

+— finally

↓

Always Execute

Important Syntax

```
try:
   # risky code

except ValueError:
   # handle error
else:
   # if successful
finally:
   # always execute
```

39. Important Keywords

3DWEBSOFT FOUNDATION

Keyword	Meaning
try	Test risky code
except	Handle exception
else	Execute if no exception
finally	Always execute
raise	Manually generate exception
Exception	General exception class

40. Knowledge Check – MCQs

1. Which block contains code that may cause an exception?

A. if
B. try
C. else
D. finally

Answer: B

2. Which block handles an exception?

A. except
B. try
C. for
D. while

Answer: A

3. Which block executes when no exception occurs?

A. except
B. else

3DWEBSOFT FOUNDATION

C. raise
D. error

Answer: B

4. Which block normally executes whether an exception occurs or not?

A. try
B. except
C. finally
D. else

Answer: C

5. What exception occurs when dividing by zero?

A. ValueError
B. TypeError
C. ZeroDivisionError
D. IndexError

Answer: C

6. Which keyword is used to manually generate an exception?

A. error
B. throw
C. raise
D. exception

Answer: C

3DWEBSOFT FOUNDATION

41. Practical Assignment

Build a **Safe Calculator Application** that:

1. Accepts two numbers.
2. Accepts an operation.
3. Performs +, -, *, /.
4. Handles invalid numbers.
5. Handles division by zero.
6. Handles invalid operations.
7. Uses finally to display:

Thank you for using the calculator.

Bonus Challenge

Add:

- Power **
- Modulus %
- Square root
- Calculation history
- JSON storage of calculation history

42. Final Takeaway

Exception handling is an essential Python skill for building **reliable applications**.

Students should remember:

try → Try the risky operation

except → Handle the problem

else → Run when successful

OJWEBSOFT FOUNDATION

finally → Always run

raise → Create an exception

3DWEBSOFT FOUNDATION